

Key Concepts in Database Management Systems (DBMS)

1. Introduction to DBMS

A Database Management System (DBMS) is software that allows users to create, store, manage, and retrieve data from a database. It acts as an interface between the user and the database, ensuring data is organized, secure, and consistent.

- **Database:** An organized collection of related data stored electronically.
- **DBMS:** Software to manage the database (e.g., MySQL, Oracle, PostgreSQL, MongoDB).
- **Data vs. Information:** Data is raw facts; information is processed, meaningful data.

2. Data Models

A data model defines how data is structured, stored, and accessed. The major types are:

- **Hierarchical Model:** Data is organized as a tree (parent-child). Fast but inflexible.
- **Network Model:** Data is organized as a graph. More flexible than hierarchical.
- **Relational Model:** Data is stored in tables (relations). Most widely used. Based on set theory.
- **Object-Oriented Model:** Data is stored as objects, like in OOP languages.
- **ER Model (Entity-Relationship):** Used for database design; shows entities, attributes, and relationships.

3. DBMS Architecture (Three-Schema)

The ANSI/SPARC three-schema architecture separates how data is stored from how users see it:

- **External Level (View Level):** What individual users see. Each user can have a different view.
- **Conceptual Level (Logical Level):** The overall logical structure of the database. What data is stored and relationships.
- **Internal Level (Physical Level):** How data is physically stored on disk (files, indexes, storage structures).

This separation provides data independence: changes at one level do not affect other levels.

4. Keys in DBMS

Keys are used to uniquely identify records in a table and to establish relationships between tables.

- **Super Key:** Any set of attributes that uniquely identifies a row.

- **Candidate Key:** Minimal super key (no redundant attributes). A table can have multiple.
- **Primary Key:** The chosen candidate key used to uniquely identify rows. Cannot be NULL.
- **Foreign Key:** An attribute in one table that references the primary key of another table. Enforces referential integrity.
- **Composite Key:** A primary key made of two or more attributes.
- **Alternate Key:** Candidate keys that are not chosen as the primary key.

5. Normalization

Normalization is the process of organizing a database to reduce data redundancy and improve data integrity. It involves dividing large tables into smaller ones and defining relationships between them.

- **1NF (First Normal Form):** No repeating groups; all attributes must have atomic (single) values.
- **2NF (Second Normal Form):** Must be in 1NF; no partial dependency (non-key attribute depends on part of composite key).
- **3NF (Third Normal Form):** Must be in 2NF; no transitive dependency (non-key attribute depends on another non-key attribute).
- **BCNF (Boyce-Codd Normal Form):** Stricter version of 3NF; every determinant must be a candidate key.

6. SQL (Structured Query Language)

SQL is the standard language used to interact with relational databases. It is divided into sub-languages:

DDL (Data Definition Language) - Defines structure

- CREATE – creates tables/databases
- ALTER – modifies structure
- DROP – deletes tables/databases

DML (Data Manipulation Language) - Manipulates data

- SELECT – retrieves data
- INSERT – adds new records
- UPDATE – modifies existing records
- DELETE – removes records

DCL (Data Control Language) - Controls access

- GRANT – gives user permissions
- REVOKE – removes user permissions

TCL (Transaction Control Language) - Manages transactions

- COMMIT – saves transaction permanently
- ROLLBACK – undoes a transaction
- SAVEPOINT – sets a point to rollback to

7. Transactions and ACID Properties

A transaction is a sequence of operations performed as a single logical unit. ACID properties ensure reliable transactions:

- **Atomicity:** All operations in a transaction succeed, or none do. No partial execution.
- **Consistency:** A transaction brings the database from one valid state to another. All integrity constraints are maintained.
- **Isolation:** Concurrent transactions do not interfere with each other. Each transaction runs as if it is the only one.
- **Durability:** Once a transaction is committed, its changes persist even after a system failure.

8. Concurrency Control

When multiple transactions execute simultaneously, problems can occur. Concurrency control mechanisms prevent these issues:

Problems in Concurrent Execution

- **Dirty Read:** A transaction reads data written by another uncommitted transaction.
- **Non-Repeatable Read:** A transaction reads the same data twice but gets different values.
- **Phantom Read:** A transaction re-executes a query and finds new rows that appeared due to another transaction.
- **Lost Update:** Two transactions update the same data; one update overwrites the other.

Concurrency Control Techniques

- **Lock-Based Protocols:** Transactions lock data before accessing it. Shared locks for reads; exclusive locks for writes.
- **Two-Phase Locking (2PL):** Transactions acquire all locks before releasing any. Prevents many anomalies.
- **Timestamp-Based Protocols:** Transactions are ordered by their timestamp. Older transactions take priority.
- **Optimistic Concurrency Control:** Assume no conflict; validate before commit.

9. Indexing

An index is a data structure that speeds up data retrieval. It works like a book index, pointing to the location of data.

- **Primary Index:** Built on the primary key. Records are physically ordered by the key.
- **Secondary Index:** Built on non-primary key fields. Used for faster searching on other attributes.
- **Dense Index:** An index entry for every record in the file.

- **Sparse Index:** An index entry for only some records (e.g., one per block).
- **B-Tree / B+ Tree Index:** Most common indexing structure. Balanced tree allows fast search, insert, and delete. B+ trees store all data in leaf nodes.
- **Hash Index:** Uses a hash function to map keys to locations. Very fast for equality searches.

10. Entity-Relationship (ER) Model

The ER model is used to design the logical structure of a database before implementation.

- **Entity:** A real-world object (e.g., Student, Course). Represented as a rectangle.
- **Attribute:** A property of an entity (e.g., Name, Age). Represented as an ellipse.
- **Relationship:** An association between entities (e.g., Student enrolls in Course). Represented as a diamond.
- **Cardinality:** Defines the number of instances in a relationship: One-to-One (1:1), One-to-Many (1:N), Many-to-Many (M:N).
- **Weak Entity:** An entity that cannot be identified without a related strong entity.

Quick Revision

- **DBMS:** Software to store, manage, and retrieve data. Examples: MySQL, Oracle.
- **Data Models:** Hierarchical, Network, Relational (most used), Object-Oriented.
- **Three-Schema Architecture:** External (user view), Conceptual (logical), Internal (physical).
- **Keys:** Primary Key (unique, not null), Foreign Key (references another table), Candidate Key (minimal super key).
- **Normalization:** 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency), BCNF (all determinants are candidate keys).
- **SQL Sublanguages:** DDL (structure), DML (data), DCL (access), TCL (transactions).
- **ACID:** Atomicity, Consistency, Isolation, Durability — properties of reliable transactions.
- **Concurrency Issues:** Dirty read, non-repeatable read, phantom read, lost update.
- **Concurrency Solutions:** Locking, Two-Phase Locking (2PL), Timestamps, Optimistic Control.
- **Indexing:** Speeds up queries. Types: Primary, Secondary, Dense, Sparse, B+Tree, Hash.
- **ER Model:** Design tool using Entities, Attributes, Relationships, and Cardinality.